Bounded Structural Model Finding with Symbolic Data Constraints

Artur Boronat[1]*

University of Leicester, Leicester, UK
artur.boronat@leicester.ac.uk

Abstract. Bounded model finding is a key technique for validating software designs, usually obtained by translating high-level specifications into SAT/SMT problems. Although effective, such translations introduce a semantic gap and a dependency on external tools. We present the *Maude Model Finder* (MMF), a native approach that brings bounded model finding to the Maude rewriting logic framework. MMF provides a schema-parametric engine that repurposes symbolic reachability for structural solving, generating finite object configurations from class declarations and graph and data constraints without bespoke generators. Technically, MMF explores a constrained symbolic rewriting system over runtime states modulo an equational theory; SMT is used only to prune states by constraint satisfiability and to discharge entailment checks for subsumption and folding. We contribute a bounded, obligation-driven calculus that separates object creation from reference assignment and supports symmetry reduction by folding via Maude's ACU matching. We establish termination, soundness, and completeness of the bounded construction within the declared bounds, and justify folding via a coverage-preserving subsumption test. We focus on the calculus and its properties, illustrating it on running examples supported by a Maude prototype.

Keywords: Rewriting Logic · Maude · Bounded Model Finding · Symbolic Rewriting · Constrained Configurations · SMT

1 Introduction

Maude is a high-performance specification and analysis environment based on rewriting logic and membership equational logic [6, 7]. Its mainstream analysis capabilities target *dynamic* properties, including reachability search and LTL model checking for finite-state systems [2], and symbolic analysis for richer state spaces, including symbolic reachability and rewriting modulo SMT [3, 16]. These strengths make Maude an attractive semantic platform, but they do not directly support a complementary task that is central in modelling language engineering: bounded (structural) model finding.

* This work was supported by the European Space Agency under ESA Contract 4000133105/20/NL/AF (P-STEP project).

Bounded model finding aims to generate finite object graphs within declared bounds that satisfy (i) class and role[1] multiplicities, and (ii) structural and data constraints. Such bounded instances are essential to debug metamodels and Object Constraint Language (OCL) constraints, validate design assumptions, and obtain concrete witnesses and counterexamples. Tools such as Alloy and the USE validator provide this capability through SAT/SMT-based instance generation [11, 13]. In Maude, while one can execute specifications and explore behaviours, there is no out-of-the-box, schema-parametric mechanism dedicated to bounded *structural* instance generation for object configurations. In our earlier work on model subtyping with OCL constraints and safe reuse, bounded witnesses were obtained via the USE validator, using its Kodkod-based back end to check satisfiability of OCL constraints [4, 5]. This paper internalises bounded witness and counterexample generation within Maude.

We present the *Maude Model Finder* (MMF), a schema-parametric bounded model finding approach for Maude object configurations. MMF formulates bounded model construction as exploration of a constrained symbolic rewriting system over *runtime states* modulo oriented equations E and algebraic axioms B (such as associativity, commutativity, and identity). SMT is used only as a background solver to prune states by constraint satisfiability and to discharge entailment checks needed for subsumption and folding [3]. Concretely, we contribute: (1) a bounded, obligation-driven construction calculus that separates object allocation from reference assignment and incrementally accumulates symbolic data constraints; (2) redundancy reduction via folding, combining matching modulo ACU, shape indexing, and SMT-backed entailment; and (3) meta-theoretic guarantees for the bounded calculus, including termination, soundness, and completeness within the declared bounds, and soundness of folding with respect to semantic coverage. We focus on the calculus and its properties, illustrating it on running examples supported by a Maude prototype.

The paper is organised as follows. Section 2 presents the MMF specification style through a running example. Sections 3 and 4 introduce the semantic foundations and the symbolic construction calculus. Section 5 describes the reachability engine and its reduction pipeline, and Section 6 summarises the correctness results; full statements and proofs are available in the extended arXiv version of this paper. Section 7 reports a proof-of-concept evaluation, followed by related work and conclusions.

2 Specification with the Maude Model Finder (MMF)

We illustrate MMF on a small organizational schema with classes, references, bounds, and symbolic data constraints. An MMF specification describes a bounded family of admissible object graphs in three parts: (i) a *schema* declaring classes and roles, (ii) a *scope* fixing finite class and role bounds, and (iii) *constraints*, split into structural predicates over graphs and data invariants.

[1] A *role* is a typed reference between classes with a declared multiplicity range.

MMF generates models by *constrained symbolic rewriting* over *runtime states* $\langle C \parallel \phi \rangle$, where C is an object-graph configuration (matched modulo ACU axioms) and ϕ is an accumulated Boolean constraint over symbolic attribute variables. Exploration proceeds in two phases: an object allocation phase creates objects to meet class bounds, followed by a reference assignment phase that enumerates admissible role values within role bounds. Branches are pruned as soon as either (i) a monotonic structural error pattern is detected in the partial graph, or (ii) the constraint becomes SMT-unsatisfiable; non-monotonic global constraints are checked only on completed candidates (normal forms).

Schema and bounds. The running example has companies, employees, and projects: a `Company` has a mandatory CEO and a set of projects; an `Employee` may have a manager; a `Project` has a non-empty set of members. The schema is written in Maude's object-oriented style (Listing 1.1); role multiplicities are reflected by carrier sorts (e.g., `Oid`, `Maybe{Oid}`, `Set{Oid}`, `NeSet{Oid}`), while the effective branching space is fixed by explicit bounds. Listing 1.2 declares a finite scope via a constant `boundsDecl`: `classB` fixes class cardinalities and `roleB` constrains, per role, the admissible target class and multiplicity range.

Listing 1.1. Schema for the CEO example.

```
omod ORG-PROJ-SCHEMA is
  protecting MMF .
  class Company  | ceo : Oid , projects : Set{Oid} .
  class Project  | members : NeSet{Oid} .
  class Employee | manager : Maybe{Oid} .
endom
```

Listing 1.2. Example bounds for the CEO running example.

```
eq boundsDecl =
  classB(Company,  1, 1) ;
  classB(Employee, 2, 2) ;
  classB(Project,  0, 2) ;
  roleB(Company,  ceo,      Employee, 1, 1) ;
  roleB(Company,  projects, Project,  0, 2) ;
  roleB(Employee, manager,  Employee, 0, 1) ;
  roleB(Project,  members,  Employee, 1, 2) .
```

Structural constraints. MMF supports two structural error predicates. First, `specErrorPredGraphPartial` is evaluated on *partial* graphs to enable early pruning. It includes intrinsic well-formedness checks (e.g., unique object identifiers, no dangling edges, and adherence to upper bounds) and can be extended with user-defined monotonic error patterns; once triggered, such errors cannot be repaired by further refinement. Second, `specErrorPredGraph` is evaluated only on completed candidates (normal forms), to express non-monotonic constraints that need the full graph. In this example, we forbid a manager for the CEO and require the management relation to be acyclic (Listing 1.3). Both predicates return a value of sort `TruthPred`, whose sole constructor is `tt`. `TruthPred` is a dedicated sort distinct from Maude's `Bool`: error predicates are structurally matched

against `tt` to detect violations, whereas `Bool` is reserved for SMT constraint terms that are passed to the solver. Keeping the two sorts separate avoids ambiguity when Boolean SMT formulas are manipulated as first-class terms alongside structural graph checks.

Listing 1.3. Global constraints (cycle detection helper omitted).

```
op ceoHasManager : Configuration ~> TruthPred .
var C E1 : Oid .  var E2 : Maybe{Oid} .
var CAS AS : AttributeSet .  var CF : Configuration .
eq ceoHasManager(< C : Company | ceo : E1, CAS >
                 < E1 : Employee | manager : E2, AS > CF) = tt .
eq specErrorPredGraph(CF) =
  ceoHasManager(CF) ttor managerCyclic(CF) .
  --- ttor: short-circuit disjunction (tt-or) over TruthPred; managerCyclic: DFS
     ↪ cycle check (omitted)
```

Data invariants via constraints. Data-level properties are expressed by equational hooks[2] that contribute to the constraint ϕ. The hook `specInvOnCreate` is applied when an object is created; `specInvOnSetRef` is applied when a role value is committed. In the example, each employee is given a symbolic integer `level` constrained to a finite range; managers must have strictly higher rank (smaller `level`), and the CEO must be at level 0 (Listing 1.4). Here `i(O,AN)`, `b(O,AN)`, and `r(O,AN)` denote the symbolic SMT variables for the integer-, Boolean-, and real-valued attribute `AN` of object `O`, respectively (formalised in Section 4).

Listing 1.4. Equational constraints for the CEO example.

```
var E C : Oid .  var M : Maybe{Oid} .
eq specInvOnCreate(Employee, E) =
  i(E, level) >= (0).Integer and i(E, level) < (3).Integer .
ceq specInvOnSetRef(Employee, manager, E, M) =
  i(M, level) < i(E, level)
if M :: Oid .
eq specInvOnSetRef(Employee, manager, E, empty) = (true).Boolean .
eq specInvOnSetRef(Company, ceo, C, E) =
  i(E, level) === (0).Integer .
```

Execution modes. An MMF specification consists of the schema, `boundsDecl`, and the structural and data constraints above. The `find` mode enumerates valid models with `findAll`, while `findFirst` searches for a single witness:

```
reduce in MF-DRIVER-META : findAll(initModel(find)) .
```

```
reduce in MF-DRIVER-META : findFirst(initModel(find)) .
```

If `findFirst` yields no solution (or `findAll` an empty set), the specification is unsatisfiable within the declared bounds.

[2] We use *equational hook* to mean a Maude equation, named by convention, that fires on a specific construction event and returns a Boolean constraint contribution. This is distinct from Maude's built-in hook mechanism for foreign function interfaces.

Counterexample search (`check`). The `check` mode reuses the same bounded construction semantics and pruning, but it searches for a counterexample instead of valid models. A counterexample is an admissible normal form that violates a user-declared mixed property combining a graph pattern and attribute constraints. Graph patterns are executable predicates over the graph component and, when monotonic, can be evaluated on partial graphs to prune early. Attribute violations are searched by conjoining the negation of a constraint into ϕ; a satisfiable final constraint witnesses a violating valuation. We illustrate `check` with two small properties for the CEO example. The first property is structural and detects a non-CEO employee without a manager (Listing 1.5).

Listing 1.5. Graph property for `check`: non-CEO employee without manager.

```
op nonCeoWithoutManager : Configuration ~> TruthPred .
ceq nonCeoWithoutManager(< C : Company  | ceo : CEO, CAS >
     < E : Employee | manager : empty, AS > CF) = tt if E =/= CEO .
eq errorPredPropGraph(CF) = nonCeoWithoutManager(CF) .
```

Semantically, this graph property corresponds to an invariant $\mathsf{Inv}^{\mathsf{prop}}_{\mathsf{graph}}$. Operationally, `check` uses `errorPredPropGraph` as an executable error predicate, which returns `tt` precisely when $\mathsf{Inv}^{\mathsf{prop}}_{\mathsf{graph}}$ is violated.

The second is an attribute property that negates the intended level range and searches for a satisfying violation (Listing 1.6).

Listing 1.6. Attribute-level property for `check`: level range (searched as a violation).

```
op propInvOnCreate : ClassId Oid -> Boolean .
eq propInvOnCreate(Employee, E) =
  i(E, level) >= (0).Integer and i(E, level) < (3).Integer .
```

3 Semantic Foundations

We separate the *declarative meaning* of a specification from its *operationalization* by the MMF calculus.

Definition 1 (Core model specification). *A* core model specification $\mathcal{M} = (\mathcal{S}, \mathsf{Inv}^{\mathsf{part}}_{\mathsf{graph}}, \mathsf{Inv}^{\mathsf{full}}_{\mathsf{graph}}, \mathsf{Inv}_{\mathsf{attr}})$ *defines the validity criteria for constrained model instances. Here* $\mathcal{S}$ *is a Maude schema module declaring classes, attributes, and subclass relations;* $\mathsf{Inv}^{\mathsf{part}}_{\mathsf{graph}}$ *and* $\mathsf{Inv}^{\mathsf{full}}_{\mathsf{graph}}$ *are graph invariants for partial and completed graphs, respectively, with* $\mathsf{Inv}^{\mathsf{part}}_{\mathsf{graph}}$ *required to be monotonic; and* $\mathsf{Inv}_{\mathsf{attr}}$ *is the attribute invariant.*

We view $\mathsf{Inv}^{\mathsf{part}}_{\mathsf{graph}}$ as comprising an *intrinsic well-formedness* condition together with schema-provided monotonic constraints. Concretely, the intrinsic component requires that (i) object identifiers are unique, (ii) for every reference role, the current reference set does not exceed the declared upper multiplicity, and (iii) the given bounds are consistent, in the sense that every target class has an upper scope large enough to satisfy the maximum lower multiplicity demanded by any incoming role.

Let G range over Maude object configurations—multisets of object records $\langle o : C \mid \mathrm{AS} \rangle$ under multiset union modulo ACU. We write $G : \mathcal{S}$ to denote that G *conforms* to the schema, meaning that every object and reference in G respects the declarations and typing in $\mathcal{S}$.

Remark 1 (Invariants vs. error predicates). At the semantic level, $\mathsf{Inv}^{\mathsf{part}}_{\mathsf{graph}}$ and $\mathsf{Inv}^{\mathsf{full}}_{\mathsf{graph}}$ are invariants. Operationally, MMF's *error predicates* return tt when an invariant is *violated*. For partial graphs, this check combines built-in and schema-provided monotonic constraints. Thus, pruning checks the *negation* of semantic invariants.

Definition 2 (Bounded model specification). *A* bounded model specification $\mathcal{M}_b = (\mathcal{M}, b)$ *consists of a core specification* $\mathcal{M}$ *together with a scope* b*. The scope assigns to each class* C *a finite cardinality interval* $[\ell_C, u_C]$ *and to each role* $r : C \to D$ *a finite multiplicity interval* $[\ell_r, u_r]$*, bounding the number of targets that each object of class* C *may have along* r*.*

We write $G : \mathcal{S}_b$ iff $G : \mathcal{S}$ and all bounds in b hold in G: each class C occurs in G with cardinality in $[\ell_C, u_C]$, and for each object $o : C$ the set of target object identifiers assigned to role r of o, denoted $o.r$, has cardinality in $[\ell_r, u_r]$.

Definition 3 (Constrained model instance). *A* constrained model instance *is a pair* $M = \langle G, \rho \rangle$ *where* G *is a* graph topology *(a finite set of typed object records with role assignments, i.e., an object configuration* $G : \mathcal{S}$*) and* ρ *is a ground valuation for the attribute-level symbols occurring in* G *and in the attribute invariant* $\mathsf{Inv}_{\mathsf{attr}}(G)$*.*

Definition 4 (Conformance). *Let* $\mathcal{M}_b$ *be a bounded specification. A constrained model instance* $M = \langle G, \rho \rangle$ conforms *to* $\mathcal{M}_b$*, written* $M : \mathcal{M}_b$*, iff:*

$$G : \mathcal{S}_b \;\wedge\; (G \models \mathsf{Inv}^{\mathsf{part}}_{\mathsf{graph}}) \;\wedge\; (G \models \mathsf{Inv}^{\mathsf{full}}_{\mathsf{graph}}) \;\wedge\; (\rho \models \mathsf{Inv}_{\mathsf{attr}}(G)).$$

Definition 5 (Bounded semantics). *Define* $[\![\mathcal{M}_b]\!] \;=\; \{\, M \mid M : \mathcal{M}_b \,\}$.

4 Symbolic State and Transition Semantics

We now turn to the formal machinery enabling automatic model finding, the *MMF calculus*. Unlike operational semantics that evolves a system over time, MMF employs a *constructive* symbolic transition system where configurations denote partial graphs and transitions represent refinement decisions. This design incrementally builds the graph, explicitly separating *existence* from *topology*.

This section formalizes the symbolic transition semantics of MMF. We first define runtime states in Section 4.1, which encapsulate the partial graph and symbolic constraints. We then detail the two-phase construction process: the object generation phase in Section 4.3, which allocates objects and their data attributes, and the graph creation phase in Section 4.4, which enumerates reference topologies over the fixed object universe.

4.1 States in the MMF Runtime

The MMF calculus operationalises the bounded satisfiability problem for a bounded model specification $\mathcal{M}_b$ as a finite reachability search over runtime states. Let `Configuration` be the Maude configuration sort, i.e., multisets of objects and control tokens under multiset union modulo ACU. We write C, C' : `Configuration`. We write $\mathsf{graph}(G)$ for the object-graph fragment (a multiset of object records), and ϕ for Boolean constraints over the background SMT theories. Class and role identifiers are K and R; object identifiers O are drawn from a finite pool bounded by b.

Definition 6 (Runtime states and graph projection). *A* runtime state *is a constrained configuration $S \equiv \langle\, C \,\|\, \phi \,\rangle$, where C is a multiset of objects including* $\mathsf{graph}(G)$ *and control tokens modulo ACU, and ϕ is a Boolean constraint. Let $\pi(C)$ be the* graph projection *that erases all runtime tokens from C and keeps only the object records.*

Semantically, S denotes a set of constrained model instances $M = \langle \pi(C), \rho \rangle$ such that $\rho \models \phi$; its denotation is given in Definition 11. Object identifiers are treated as fixed uninterpreted symbols during construction; only attribute-level symbols are assigned values by the constraint valuations. A configuration C packages both the evolving graph and bounded construction control. Its core tokens include a *phase marker* $\mathsf{phase}(q)$ with $q \in \{\mathsf{objectBuild}, \mathsf{refBuild}\}$; an *identifier universe* $\mathsf{allIds}(OS)$ indexed by class; an *allocation cursor* $\mathsf{freshIds}(OL)$ and the set of allocated identifiers $\mathsf{takenIds}(TI)$; *lower-bound obligations* $\mathsf{missing}(K, n)$ and *feasibility obligations* $\mathsf{need}(K, n)$; and a *reference-task pool* $\mathsf{refPool}(RTL)$ discharged in phase $\mathsf{refBuild}$. These tokens are operational devices: they ensure boundedness, phase discipline, and exhaustive enumeration.

Remark 2 (Role of runtime tokens). Runtime tokens (identifier pools, obligation counters, task lists, phase markers) are part of C because they drive the operationalization, support termination, and enable completeness by systematic enumeration. They do *not* appear in the semantic satisfaction relation $\models$; the link is mediated by the projection π and the state denotation $[\![\cdot]\!]$.

4.2 Overview of the MMF Construction Calculus

The MMF calculus is *two-phase*: in phase $\mathsf{objectBuild}$ it allocates a bounded set of objects, and in phase $\mathsf{refBuild}$ it assigns reference-valued attributes. Let B be the structural axioms involved in the constructors of $\mathcal{M}$ and runtime tokens. Each one-step move is a rule application on the runtime configuration C modulo B, followed by equational normalization by the oriented equations $\boldsymbol{E}$ modulo B.

We distinguish three layers of rewriting: a *raw* rule step operates on the configuration C (modifying the graph topology G); a *normalized* step applies equational simplification; and a *constrained* step lifts the normalized structural transition to the level of runtime states by accumulating the attribute constraints implied by the structural change.

Definition 7 (Raw rewriting step). *We write $C \rightarrow^{\ell,\sigma}_{R/B} C_1$ iff there exists a rule in R with label ℓ and a substitution σ (obtained by matching modulo B) such that applying the instantiated rule $\ell\sigma$ rewrites C to C_1 modulo B.*

Definition 8 (Normalized rewriting step). *We write $C \Rightarrow^{\ell,\sigma}_{R,\boldsymbol{E}/B} C'$ iff there exists an intermediate C_1 such that $C \rightarrow^{\ell,\sigma}_{R/B} C_1$ and $C' = C_1 \downarrow_{\boldsymbol{E}/B}$, where $C_1 \downarrow_{\boldsymbol{E}/B}$ denotes the canonical form of C_1 with respect to the oriented equations $\boldsymbol{E}$ modulo axioms B.*

Remark 3 (Maude implementation of rewriting steps). The raw step $C \rightarrow^{\ell,\sigma}_{R/B} C_1$ is realised via Maude's reflective `metaApply`, which applies a single rewrite rule and returns the result modulo B. Normalization $\downarrow_{\boldsymbol{E}/B}$ corresponds to `metaReduce`, which computes the $\boldsymbol{E}/B$-canonical form. These are the standard reflective operations used throughout the MMF engine (Section 5).

Only two rule families add user attribute constraints to the guard: object creation (Obj-Gen) and reference commitment (Ref-Choose). We factor these event-specific contributions through two hook shorthands:

$$\mathsf{hook}_{\mathsf{create}}(K, o) \triangleq \mathsf{specInvOnCreate}(K, o)$$
$$\mathsf{hook}_{\mathsf{setref}}(K, R, o, v) \triangleq \mathsf{specInvOnSetRef}(K, R, o, v)$$

and define the auxiliary function $\mathsf{hook}_\ell(\sigma)$ to capture the rule-indexed injection logic:

$$\mathsf{hook}_\ell(\sigma) \triangleq \begin{cases} \mathsf{hook}_{\mathsf{create}}(K\sigma, O\sigma) & \text{if } \ell = \textsc{Obj-Gen} \\ \mathsf{hook}_{\mathsf{setref}}(K\sigma, R\sigma, O\sigma, v\sigma) & \text{if } \ell = \textsc{Ref-Choose} \\ \mathsf{true} & \text{otherwise} \end{cases}$$

Definition 9 (Constrained rewriting step). *We write*

$$\langle\, C \,\|\, \phi \,\rangle \Rightarrow^{\ell,\sigma}_{R,\boldsymbol{E}/B} \langle\, C' \,\|\, \phi' \,\rangle \qquad \textit{if} \qquad C \Rightarrow^{\ell,\sigma}_{R,\boldsymbol{E}/B} C',$$

and the constraint is strengthened by the corresponding hook and normalized: $\phi' = \mathsf{simpB}\big(\phi \wedge \mathsf{hook}_\ell(\sigma)\big) \downarrow_{\boldsymbol{E}/B}$[3].

Remark 4 (Contextual conventions). Unless stated otherwise, all runtime configurations and constraints are maintained in $\boldsymbol{E}/B$-canonical form. In the rule presentations below, we write $\langle\, C \,\|\, \phi \,\rangle \Rightarrow \langle\, C' \,\|\, \phi' \,\rangle$ as shorthand for the constrained rewriting step $\langle\, C \,\|\, \phi \,\rangle \Rightarrow^{\ell,\sigma}_{R,\boldsymbol{E}/B} \langle\, C' \,\|\, \phi' \,\rangle$; the rule label ℓ, matching substitution σ, and theory parameters $R, \boldsymbol{E}, B$ are implicit from context. We write $S \Rightarrow S'$ when $S \Rightarrow^{\ell,\sigma}_{R,\boldsymbol{E}/B} S'$ holds for some ℓ and σ (Definition 9); in `check` mode, the step additionally updates the witness component ψ_P as described below.

[3] The simpB function implements a deterministic simplification strategy that normalizes Boolean constraints by flattening associative-commutative operators, ordering literals, pushing negations into relational atoms, performing arithmetic constant folding, and eliminating identity elements.

Let $\mathsf{init}(\mathcal{M}_b, \mathsf{mode})$ be the canonical initial runtime state. The configuration C_0 contains $\mathsf{phase}(\mathsf{objectBuild})$, an identifier universe $\mathsf{allIds}(OS)$ and allocation list $\mathsf{freshIds}(OL)$, where OL is a fixed linearization of the identifiers in OS induced by the class order $\mathsf{classList}$ and the per-class upper bounds in b. Its head determines the next identifier to be consumed and hence the canonical allocation order. It also contains the initial bounded-work obligations: for every class C in the schema, C_0 includes a token $\mathsf{missing}(C, \ell_C)$, where ℓ_C is the class lower bound from b, and a token $\mathsf{need}(C, \mathrm{inNeed}(C))$, where $\mathrm{inNeed}(C)$ is the maximum lower multiplicity among roles targeting C. Finally, $\phi_0 = \mathsf{true}$. If $\mathsf{mode} = \mathtt{find}$, then $\mathsf{init}(\mathcal{M}_b, \mathtt{find}) = \langle\, C_0 \,\|\, \phi_0 \,\rangle$. If $\mathsf{mode} = \mathtt{check}$, then $\mathsf{init}(\mathcal{M}_b, \mathtt{check}) = \langle\, C_0 \,\|\, \phi_0 \,\|\, \psi_0 \,\rangle$ with $\psi_0 = \mathsf{false}$. In $\mathtt{check}$ mode, ϕ accumulates hard constraints, as usual, but ψ_P accumulates an existential violation witness.

Property instrumentation (`check`). In $\mathtt{check}$ mode, the calculus is additionally parameterised by a mixed property with a graph component and an attribute component. Semantically, the graph component is an invariant $\mathsf{Inv}^{\mathsf{prop}}_{\mathsf{graph}}$ evaluated on projected graphs. Operationally, the implementation provides an error predicate $\mathsf{errorPredPropGraph}$ such that $\mathsf{errorPredPropGraph}(G) = \mathsf{tt}$ iff $G \not\models \mathsf{Inv}^{\mathsf{prop}}_{\mathsf{graph}}$. The attribute component is specified by hook equations $\mathsf{propInvOnCreate}$ and $\mathsf{propInvOnSetRef}$ (Section 2). Operationally, $\mathtt{check}$ reuses the same construction rules and the same hard-constraint accumulation. In addition, it records whether some property violation can be witnessed by maintaining a separate Boolean witness component ψ_P that accumulates violation evidence disjunctively across partial states. Formally, for a rule instance with label ℓ and matching substitution σ, define

$$\mathsf{propViol}_\ell(\sigma) \triangleq \begin{cases} \neg\, \mathsf{propInvOnCreate}(K\sigma, O\sigma) & \text{if } \ell = \textsc{Obj-Gen} \\ \neg\, \mathsf{propInvOnSetRef}(K\sigma, R\sigma, O\sigma, v\sigma) & \text{if } \ell = \textsc{Ref-Choose} \\ \mathsf{false} & \text{otherwise.} \end{cases}$$

In $\mathtt{check}$ mode, the hard constraint is updated as usual. The witness is updated by disjunction: $\psi'_P = \mathsf{simpB}\big(\psi_P \vee \mathsf{propViol}_\ell(\sigma)\big)$. Accordingly, in $\mathtt{check}$ mode we write

$$\langle\, C \,\|\, \phi \,\|\, \psi_P \,\rangle \Rightarrow^{\ell,\sigma}_{R,\boldsymbol{E}/B} \langle\, C' \,\|\, \phi' \,\|\, \psi'_P \,\rangle$$

iff $C \Rightarrow^{\ell,\sigma}_{R,\boldsymbol{E}/B} C'$ and the above two updates hold.

The calculus runs in two modes: $\mathtt{find}$ enumerates valid models satisfying all structural and data invariants, while $\mathtt{check}$ searches for a counterexample to a user-declared property. Normal-form acceptance is defined accordingly.

Definition 10 (Normal forms, admissibility, acceptance). *Normal form* $\mathsf{NF}(S)$ *and* $\mathsf{Admissible}(S)$ *are defined as follows:*

- $\mathsf{NF}(S)$ *holds iff no calculus rule applies to* S*, i.e., iff* $\neg\exists S'.\ S \Rightarrow S'$.
- $\mathsf{Admissible}(S)$ *holds iff* $\pi(C) \models \mathsf{Inv}^{\mathsf{part}}_{\mathsf{graph}}$ *and* $sat(\phi)$.

– $\mathsf{Accept}_{\mathtt{find}}(S)$ *holds iff* $\mathsf{NF}(S) \wedge \mathsf{Admissible}(S) \wedge \big(\pi(C) \models \mathsf{Inv}^{\mathsf{full}}_{\mathsf{graph}}\big)$.
– $\mathsf{Accept}_{\mathtt{check}}(S)$ *holds iff* $\mathsf{Accept}_{\mathtt{find}}(S)$ *and, additionally,*

$$\pi(C) \not\models \mathsf{Inv}^{\mathsf{prop}}_{\mathsf{graph}} \;\vee\; sat(\phi \wedge \psi_P).$$

Remark 5 (Counterexamples in `check` *mode).* In `check` mode, hard constraints are accumulated in ϕ and define the denotation $[\![S]\!]$. Property violations are accumulated in the witness component ψ_P by disjunction. Therefore, if $sat(\phi \wedge \psi_P)$ holds, there exists a valuation ρ such that $\rho \models \phi$ and $\rho \models \psi_P$. Then $\langle \pi(C), \rho \rangle \in [\![S]\!]$. Moreover, under ρ the attribute-level property is violated at least once. Graph-level violations are witnessed independently by $\pi(C) \not\models \mathsf{Inv}^{\mathsf{prop}}_{\mathsf{graph}}$.

Definition 11 (State semantics). *Let ρ range over valuations that assign* ground *values to all attribute-level symbols occurring in ϕ (e.g., symbols of the form $i(O, AN)$, $r(O, AN)$, $b(O, AN)$ for an object identifier O and attribute name AN), while leaving the object identifiers occurring in C unchanged. Define the* semantics *of a runtime state by*

$$[\![\langle C \,\|\, \phi \rangle]\!] \;=\; \{\, \langle \pi(C), \rho \rangle \mid \rho \models \phi \,\} \;=\; [\![\langle C \,\|\, \phi \,\|\, \psi_P \rangle]\!].$$

Intuitively, $[\![\langle C \,\|\, \phi \rangle]\!]$ is the set of constrained model instances $M = \langle \pi(C), \rho \rangle$ such that ρ satisfies the accumulated constraint ϕ. The projection $\pi(C)$ provides the graph topology and the object identifiers, which are treated as fixed uninterpreted symbols during construction. The valuation ρ supplies ground values for the attribute-level symbols occurring in ϕ. In `check` mode, the witness component ψ_P affects acceptance but not denotation.

4.3 Object Generation Phase (objectBuild)

Phase objectBuild allocates fresh objects and initializes reference-assignment obligations. We assume mode `find` and a configuration containing phase(objectBuild). The transition relation $\Rightarrow$ is given by constrained rewriting rules over runtime states. Rule Obj-Gen matches the configuration modulo ACU to extract the next identifier O_K from $\mathsf{freshIds}(O_K :: OL)$ (thus fixing a canonical allocation order). It consumes O_K, creates a fresh object, and decrements the class-obligation counters $\mathsf{missing}(K, N)$ and $\mathsf{need}(K, L)$. Here $\ominus$ denotes saturating subtraction, i.e., $n \ominus 1 = \max(n-1, 0)$.

Object creation performs (i) *structural initialization* and (ii) *scheduling of later reference choices*: $\mathsf{initAttrs}(K, O_K)$ populates attributes with well-typed placeholders (e.g., $\emptyset$ for sets and a distinguished value for single references), while addTasks enqueues one task $\mathsf{refIdx}(O_K, R, K', 0, REM)$ per role $R : K \to K'$, where 0 is the initial rank and REM is a remaining-rank budget that represents the (finite) bounds-induced choice space. The constraint is strengthened with the user hook $\mathsf{specInvOnCreate}(K, O_K)$ and canonicalized by simpB.

$$\textsc{Obj-Gen}\quad \langle \begin{array}{l}\mathsf{graph}(G),\\ \mathsf{freshIds}(O_K :: OL),\\ \mathsf{takenIds}(TI),\\ \mathsf{missing}(K,N),\\ \mathsf{need}(K,L),\\ \mathsf{refPool}(RTL)\ldots\end{array} \parallel \phi \rangle \;\Rightarrow\; \langle \begin{array}{l}\mathsf{graph}(G,\ new),\\ \mathsf{freshIds}(OL),\\ \mathsf{takenIds}(O_K,\ TI),\\ \mathsf{missing}(K, N \ominus 1),\\ \mathsf{need}(K, L \ominus 1),\\ \mathsf{refPool}(RTL')\ldots\end{array} \parallel \phi' \rangle$$

where

$$\begin{aligned}\phi' &= \mathsf{simpB}(\phi \wedge \mathsf{hook}_{\mathsf{create}}(K, O_K)),\\ RTL' &= \mathsf{addTasks}(O_K, K, \mathsf{roleList}(K), \mathsf{allIds}(OS), RTL),\\ new &= \langle O_K : K \mid \mathsf{initAttrs}(K, O_K, \mathsf{allIds}(OS))\rangle.\end{aligned}$$

Rule Obj-Skip allows exploring graphs smaller than the maximum scope. It applies only when lower-bound obligations are met ($\mathsf{missing}(K,0)$ and $\mathsf{need}(K,0)$):

$$\textsc{Obj-Skip}\quad \langle \begin{array}{l}\mathsf{freshIds}(O_K :: OL),\\ \mathsf{missing}(K,0),\\ \mathsf{need}(K,0)\ldots\end{array} \parallel \phi \rangle \;\Rightarrow\; \langle \begin{array}{l}\mathsf{freshIds}(OL),\\ \mathsf{missing}(K,0),\\ \mathsf{need}(K,0)\ldots\end{array} \parallel \phi \rangle$$

where the constraint is unchanged and the allocation cursor advances by dropping O_K.

Remark 6 (Partial states and global invariants). Intermediate states during objectBuild are *partial*: they do not necessarily satisfy non-monotonic global invariants $\mathsf{Inv}^{\mathsf{full}}_{\mathsf{graph}}$. Only monotonic constraints (captured by specErrorPredGraphPartial) are checked during construction. Full invariants are verified only at acceptance.

The objectBuild phase consists of the repeated application of these symbolic transition rules until freshIds(nil) is exhausted. This exhaustion triggers the transition to the refBuild phase. Crucially, as the calculus switches phases, it restricts the global $\mathsf{allIds}(OS)$ component to exactly the set of identifiers present in takenIds. This filtering step ensures that the subsequent reference generation phase considers only the objects that were actually allocated, preventing the creation of dangling edges to skipped or non-existent objects by construction.

4.4 Graph Creation Phase (refBuild)

In the refBuild phase, reference tasks are discharged. A task $RT = \mathsf{refIdx}(O_K, R, K', \mathsf{curr}, REM)$ stores a *rank cursor* curr together with a *remaining-rank budget* REM. It represents the interval of admissible ranks $[\mathsf{curr}, \mathsf{curr} + REM]$ in the canonical enumeration of role values.

A key semantic choice in refBuild is that role values are not generated by incremental extension, but are instead selected as elements of a finite, *intrinsically defined* domain determined solely by the declared multiplicity bounds and

the identifier universe. Fix a reference task $\mathsf{refIdx}(O_K, R, K', \mathsf{curr}, REM)$ and let OS be the class-indexed identifier set associated with K' in $\mathsf{allIds}(OS)$. Write $P = [o_1, \ldots, o_n]$ for the canonical list obtained from OS under a fixed total order on identifiers, and let ℓ, u be the lower and upper multiplicity bounds for role R (with $0 \leq \ell \leq u \leq n$). The set $\mathcal{D}_{\ell,u}(OS)$ of all admissible reference assignments is the union of all subsets of targets whose size falls within the declared bounds:

$$\mathcal{D}_{\ell,u}(OS) \;=\; \bigcup_{m=\ell}^{u} \{\, S \subseteq OS \mid |S| = m \,\}, \qquad U_{\max} \;=\; |\mathcal{D}_{\ell,u}(OS)| - 1.$$

Initially, a task for this role is created with $\mathsf{curr} = 0$ and $REM = U_{\max}$, so that the represented rank interval is exactly $[0, U_{\max}]$. MMF induces a canonical enumeration over this space without materializing it. The order is defined hierarchically: first by subset size m (from ℓ to u), and then, within each size, by lexicographic unranking induced by the list order P.

The core mechanism is the unranking function $\mathsf{unrank}(P, m, j)$, which acts as a stateless cursor. It reconstructs the j-th subset of size m directly from P using the standard combinatorial number system for ranking and unranking combinations (see, e.g., [12]). The intrinsic selector $\mathsf{choiceRef}$ is then defined by delegating to the ranked enumeration:

$$\mathsf{choiceRef}(OS, K, R, K', \mathsf{curr}) \;=\; \mathsf{rankedFromTo}(P, \ell, u, \mathsf{curr}).$$

Rule Ref-Choose triggers a symbolic transition that commits to the assignment defined by the current rank curr. It calculates the reference v using $\mathsf{choiceRef}$ and performs the semantic update to the object's attribute set:

Ref-Choose

$$\begin{array}{l} \mathsf{allIds}(OS) \\ \langle\, \mathsf{graph}(\langle O_K : K \mid \mathrm{AS}\rangle \ldots),\ \|\, \phi \,\rangle \\ \mathsf{refPool}(RT :: RTL) \ldots \end{array} \;\Rightarrow\; \begin{array}{l} \mathsf{allIds}(OS) \\ \langle\, \mathsf{graph}(\langle O_K : K \mid \mathrm{AS}'\rangle \ldots),\ \|\, \phi' \,\rangle \\ \mathsf{refPool}(RTL) \ldots \end{array}$$

where $RT = \mathsf{refIdx}(O_K, R, K', \mathsf{curr}, REM)$, $v = \mathsf{choiceRef}(OS, K, R, K', \mathsf{curr})$, $\mathrm{AS}' = \mathsf{setRef}(\mathrm{AS}, R, v)$, and $\phi' = \mathsf{simpB}(\phi \wedge \mathsf{hook}_{\mathsf{setref}}(K, R, O_K, v))$.

Rule Ref-Skip represents the decision to *discard* the current assignment and continue searching. It increments the cursor curr to explore the next candidate and decrements the remaining-rank budget REM, thereby shrinking the residual rank interval while preserving the global upper rank $\mathsf{curr} + REM$.

$$\textsc{Ref-Skip} \quad \langle\, \mathsf{refPool}(RT :: RTL) \ldots \|\, \phi \,\rangle \;\Rightarrow\; \langle\, \mathsf{refPool}(RT' :: RTL) \ldots \|\, \phi \,\rangle$$

where $RT = \mathsf{refIdx}(O_K, R, K', \mathsf{curr}, REM)$, $REM > 0$, and $RT' = \mathsf{refIdx}(O_K, R, K', \mathsf{curr}{+}1, REM{-}1)$. Repeated application of Ref-Skip advances through the bounds-induced domain of admissible values. When $REM = 0$, the only option is to choose (or fail if the domain is empty), ensuring termination. Completion of $\mathsf{refBuild}$ yields a normal form NF, checked for global acceptance.

Remark 7 (Finite enumeration). Each reference task has a finite choice space determined by b, so termination follows: Ref-Skip decreases the remaining interval width, and Ref-Choose removes the task from the pool.

5 Symbolic Reachability Engine

Section 4 defined the symbolic transition relation $\Rightarrow$ over runtime states $\langle C \parallel \phi \rangle$. We now define a meta-level exploration strategy that traverses $\Rightarrow$ under a fixed reduction pipeline comprising exact deduplication, admissibility pruning, and optional folding. The exploration engine maintains a loop state

$$L = (PQ,\ \mathit{SeenMap},\ \mathit{SeenSuccMap},\ \mathit{ExactMap},\ \mathit{Profiler}),$$

organising the exploration components. The priority queue PQ serves as a worklist of pending nodes, where each node packages a runtime state with an index key and a cached NF flag. Heuristic search and folding are facilitated by *SeenMap*, which stores retained representatives per index key for coverage-based pruning, and *SeenSuccMap*, which memoises exact successor states to suppress duplicates. Finally, *ExactMap* collects accepted normal-form states as they are found, while the *Profiler* tracks instrumentation counters.

5.1 Meta-level Exploration Semantics

The search procedure is parameterized by a configuration term `Opt`, which controls active reduction strategies (e.g., symmetry handling, folding). Starting from an initial state S_0, the loop initializes the state components L and iterates as long as the worklist is non-empty. In each step, the engine selects a pending node via Select and delegates it to processNode. The loop terminates when the frontier is exhausted or, in `findFirst` mode, as soon as a witness is recorded.

The processNode routine bifurcates based on the state's status. If S is a normal form ($\mathsf{NF}(S)$), it constitutes a candidate result; the engine validates it via the mode-specific acceptance predicate and, if successful, records it in *ExactMap*. If S is expandable, the engine computes its one-step symbolic successors using the expansion operator Step(S) (implemented via Maude's reflective `metaApply`). These raw successors then traverse the reduction pipeline: FilterExact eliminates duplicates against *SeenSuccMap*; Prune discards inadmissible states (violating $\mathsf{Inv}^{\mathsf{part}}_{\mathsf{graph}}$ or with an unsatisfiable constraint); and, if enabled by `Opt`, a folding stage enforces semantic coverage against *SeenMap*. In the current implementation, folding is applied only to successors that are already in normal form. Finally, surviving successors are added to the worklist via Insert.

5.2 Subsumption and Folding

This subsection formalizes the redundancy elimination mechanism used by the exploration loop. The key idea is to maintain, among explored normal forms, a

covering antichain with respect to a *semantic embedding preorder*. This preorder soundly approximates the semantic *coverage* relation and supports both queue-time pruning (FoldSift) and repository maintenance (FoldRefresh).

Recall from Definition 11 that a runtime state $S \equiv \langle\, C \,\|\, \phi \,\rangle$ denotes a set $[\![S]\!]$ of constrained model instances. Each constrained model instance is a pair $M = \langle \pi(C), \rho \rangle$ consisting of a graph topology and a satisfying valuation. We recall the *semantic coverage preorder* $\preceq$:

$$S_1 \preceq S_2 \quad \text{iff} \quad [\![S_2]\!] \subseteq [\![S_1]\!].$$

Intuitively, S_1 covers S_2 if every constrained model instance represented by S_2 is also represented by S_1. Coverage is a preorder, not an equivalence. The engine realizes the preorder $\preceq_{\mathrm{MMF}}$ via $\texttt{subsumes}(S_1, S_2)$, using three criteria to determine coverage. First, an *index equality* gate checks $\eta(S_1) = \eta(S_2)$ to filter incompatible candidates. Second, *structural embedding* employs `metaMatch` to find a substitution σ such that $\pi(C_2) =_{E \cup B} \pi(C_1)\sigma$. Third, *constraint entailment* verifies via SMT that ϕ_2 entails $\phi_1\sigma$. Meeting these guarantees S_1 represents every constrained model instance of S_2, making `subsumes` a sound (albeit incomplete) semantic coverage test.

Folding maintains a covering antichain $\mathcal{A}$ of normal-form representatives during exploration. Thus, $\mathcal{A}$ contains no mutually subsuming representatives yet covers all explored normal forms. Operationally, FoldSift discards a candidate S only if $\mathsf{NF}(S)$ and some $R \in \mathcal{A}$ satisfies $\texttt{subsumes}(R, S)$. Conversely, FoldRefresh inserts new representatives by removing any $R \in \mathcal{A}$ subsumed by the incoming state. Hence, coverage is sound: every constrained model instance in the bounded semantics remains represented by at least one retained representative in the exploration graph. MMF does not compute graph isomorphism modulo arbitrary identifier permutations. Instead, symmetry reduction comes from representing configurations modulo the equational theory $(\Sigma, E \cup B)$ (including ACU for multiset composition) and from shape indexing. Within a bucket, structural embedding is decided by matching modulo $E \cup B$ via `metaMatch`, while data-level constraints in ϕ are discharged by SMT.

6 Correctness of the MMF Calculus

We summarise the main metatheoretic properties of the bounded construction calculus and its search engine. The bounded model specification $\mathcal{M}_b$ fixes a finite universe and role ranges, so all guarantees are relative to the declared bounds. Soundness ensures that any accepted normal form denotes only constrained model instances that conform to $\mathcal{M}_b$, so reported witnesses are never spurious. Completeness ensures that every constrained model instance admitted by $\mathcal{M}_b$ is represented by some retained accepted state, so folding and subsumption do not discard valid solutions. Together, these results show that the calculus explores the entire bounded search space modulo coverage-preserving reductions.

Termination and well-formedness explain why exploration is finite and why intermediate states remain meaningful. Runtime well-formedness $\mathsf{WF}(S)$ cap-

run	ms	rewrites	popped	NF	pruned	folded
`findAll(find)`	2181	7.92M	450	52	354	88
`findFirst(find)`	136	284.8K	29	1	27	0
`findAll(check)`	2409	11.11M	374	40	298	64
`findFirst(check)`	144	317.6K	29	1	21	0

Table 1. Mode comparison under the baseline pipeline. *pruned* = `prunedBad`+`prunedUnsat`.

tures intrinsic structural consistency of partial graphs and the correct accumulation of constraint obligations in ϕ, and each rule preserves it. Under standard assumptions on equational normalization and finite enumeration of reference tasks, the lexicographic measure on remaining identifiers and task widths strictly decreases. Consequently, the bounded search terminates and pruning by monotonic structural constraints and SMT unsatisfiability remains correct. full statements and proofs are available in the extended arXiv version of this paper.

7 Evaluation

We evaluate MMF on the CEO benchmark (Company–Employee–Project) from Section 2, which combines structural constraints (e.g., "CEO has no manager", acyclicity) with symbolic integer invariants. The goal is a proof-of-concept assessment of feasibility rather than an exhaustive performance study. Tables 1 and 2 use the bounds from Listing 1.2: 1 company, 2 employees, and 0–2 projects. Table 3 varies the employee bound from 2 to 5. Timeouts are set at 120 s. All runs use a deterministic worklist policy (`useHeuristicPQ=false`). We report wall-clock time (`ms`), Maude rewrites, and driver counters: expanded states (`popped`), normal forms (`NF`), successors generated/enqueued, and eliminations by exact deduplication, structural pruning (`prunedBad`), SMT unsatisfiability (`prunedUnsat`), and folding (`folded`). Unless stated otherwise, the baseline enables SMT pruning, folding, and shape indexing (`useSymbContraintFiltering=true`, `useFolding=true`, `useIndex=true`), and removes redundant solutions at insertion (`removeRedundantSolutions=true`).

Table 1 compares `findAll`/`findFirst` and `find`/`check` under the baseline pipeline. For `findAll`, more than half of generated successors are eliminated before queueing, with structural pruning being the largest contributor and SMT pruning and folding providing additional reductions. `findAll(check)` expands fewer states than `findAll(find)` but is slightly slower, indicating higher per-state overhead in `check` (property-side evaluation and constraint processing). For `findFirst`, folding does not contribute on this scope (`folded`=0); runtime is largely determined by pruning and successor generation.

Table 2 isolates the reduction mechanisms for `findAll(find)`. Disabling SMT pruning while keeping folding enabled times out, whereas disabling both SMT pruning and folding completes but is an order of magnitude slower than baseline; thus SMT satisfiability filtering is essential and also stabilizes folding

run	ms	rewrites	popped	NF	inserted	redundant
`A1-baseline`	2185	7.92M	450	52	52	0
`A2-noSMT`	timeout	-	-	-	-	-
`A3-noFolding`	1258	2.20M	494	96	52	44
`A4-folding-noIndex`	timeout	-	-	-	-	-
`A5-noSMT-noFolding`	20299	28.01M	2054	192	100	92

Table 2. Ablations for `findAll(find)`. Timeouts occur at 120 s.

employees	ms (base)	rewrites (base)	folded (base)	ms (noFold)	rewrites (noFold)
2	97	0.10M	0	88	0.05M
3	147	0.32M	0	107	0.05M
4	737	4.08M	3	172	0.13M
5	3098	20.27M	6	264	0.24M

Table 3. Scalability for `findFirst(check)` as the employee bound increases.

(which relies on entailment checks). Disabling folding decreases runtime but increases the number of normal forms; redundant solutions are then filtered only at insertion time, showing that folding prevents exploring many redundant candidates to completion. Folding without indexing times out, indicating that shape indexing is a prerequisite for making folding feasible.

Table 3 scales `findFirst(check)` from 2 to 5 employees, comparing baseline to `noFolding`. Baseline time increases sharply with the bound, while `noFolding` scales much more gently; at 5 employees, `noFolding` is over an order of magnitude faster. Since folding eliminates only a few successors at these bounds, this suggests that subsumption checks dominate cost in witness-oriented search when folding successes are rare. All `folding-noIndex` scalability runs timed out, consistent with the ablation results: indexing is necessary for folding to be viable.

Overall, structural pruning removes most successors in exhaustive search, while SMT pruning is essential for feasibility and stabilises folding. Folding reduces redundant candidates when shape indexing makes subsumption checks practical. In `check` mode, witness search incurs higher per-state overhead but remains practical on the explored bounds.

The evaluation focuses on one benchmark family; folding effectiveness may differ for specifications with stronger symmetries or weaker constraints. We report single-run timings; although runs are deterministic, wall time can vary with system load. Rewrite counts are a stable internal proxy for work but do not isolate SMT solver time. Timeout-based conclusions depend on the 120 s cutoff, but the observed blow-ups are consistent across the "noSMT+folding" and "folding-noIndex" configurations.

8 Related work

Bounded Model Finding. MMF shares the goal of bounded model finders like Alloy [11] and USE [10,13]: generating concrete object graphs that satisfy structural

constraints. Unlike these tools, which typically rely on translation to SAT/SMT solvers (often creating a semantic gap), MMF operates as a native symbolic rewriting engine within rewriting logic. This allows domain logic and constraints to be defined directly in Maude [7], providing a uniform environment for specification, analysis, and verification.

Symbolic Reachability and Symbolic Rewriting. Symbolic analysis in Maude ranges from narrowing-based reachability to symbolic rewriting modulo SMT [3, 14, 16]. MMF repurposes this symbolic infrastructure for bounded structural model finding. Technically, MMF explores a constrained symbolic rewriting system over runtime states that refine partial object graphs, rather than using narrowing to solve for substitutions that drive temporal reachability. The correctness of our calculus rests on established results in this domain. Our soundness result mirrors the simulation results of Bae and Rocha [3] and Arias et al. [1], ensuring that every symbolic step corresponds to a valid refinement of the concrete model space. Our completeness result follows from a bounded two-phase enumeration argument within the user-declared bounds, guaranteeing that our generation strategy can construct any satisfying concrete model instance that exists within those bounds.

State Space Folding. To ensure feasibility, MMF employs state folding to prune redundant search paths. This technique relies on a subsumption relation that approximates semantic containment. As formalized by Arias et al. [1] and Meseguer [15], a correctly defined subsumption relation ($\preceq$) ensures that if a state S is subsumed by an existing representative S', pruning S preserves completeness because all constrained model instances denoted by S are already covered by S'. MMF implements this via graph shape indexing and logical entailment checks.

Comparison with specific approaches. Riesco's testing techniques exploit Maude's symbolic engines (notably narrowing) to generate concrete witnesses [17]. However, Riesco targets test input generation for functional correctness of operational semantics, whereas MMF focuses on structural model finding under schema constraints, using SMT integration primarily for satisfiability of data invariants rather than full symbolic execution. Escobar et al. develop folding optimisations for SMT-based symbolic reachability based on narrowing [9]. MMF adapts these principles but specializes the redundancy layer (shape indexing, ACU-matching, and entailment-based subsumption) to exploit order-only symmetries induced by the equational theory (notably ACU) and to reduce redundancy in strictly bounded structural generation.

9 Conclusion and Future Work

This paper presented the Maude Model Finder (MMF), a native approach to bounded model finding in the Maude ecosystem. By formalizing bounded model generation as symbolic transition exploration over runtime states within rewriting logic, MMF allows users to validate structural and data-level constraints

without departing from Maude's framework. Our approach synthesizes the structural generation techniques of SAT-based finders with symbolic rewriting modulo SMT, using SMT strictly as a background solver for constraint satisfiability and entailment, and leveraging folding to quotient redundant symbolic states. We establish termination, soundness, and completeness of the bounded construction within the declared bounds, and justify folding via a coverage-preserving subsumption test.

Several avenues for future work remain. First, we plan to generalize the current meta-level exploration engine by integrating Maude's strategy language [8], offering users a declarative way to customize search heuristics and pruning policies. Second, while our current symmetry reduction relies on canonical construction and post-hoc folding, we aim to investigate constructive symmetry breaking techniques to further prune the frontier. Finally, we intend to expand the supported constraint language to cover a richer subset of OCL, enhancing MMF's utility for MDE applications.

Acknowledgements

The author is grateful to José Meseguer and Santiago Escobar for insightful discussions and detailed feedback on drafts of this article.

References

1. Arias, J., Bae, K., Olarte, C., Ölveczky, P.C., Petrucci, L.: A Rewriting-logic-with-SMT-based Formal Analysis and Parameter Synthesis Framework for Parametric Time Petri Nets. Fundamenta Informaticae **192**(3-4), 261–312 (September 2024). https://doi.org/10.3233/FI-242195, publisher: SAGE Publications
2. Bae, K.: Rewriting-based model checking methods. Ph.D. thesis, University of Illinois at Urbana-Champaign (September 2014), https://hdl.handle.net/2142/50553
3. Bae, K., Rocha, C.: Symbolic state space reduction with guarded terms for rewriting modulo SMT. Science of Computer Programming **178**, 20–42 (June 2019). https://doi.org/10.1016/j.scico.2019.03.006
4. Boronat, A.: Structural model subtyping with OCL constraints. In: Proceedings of the 10th ACM SIGPLAN International Conference on Software Language Engineering, SLE 2017, Vancouver, BC, Canada, October 23-24, 2017. pp. 194–205. ACM (2017)
5. Boronat, A.: Safe reuse in modelling language engineering using model subtyping with OCL constraints. Software and Systems Modeling **22**(3), 797–818 (Jun 2023). https://doi.org/10.1007/s10270-022-01028-7
6. Clavel, M., Durán, F., Eker, S., Lincoln, P., Martí-Oliet, N., Meseguer, J., Talcott, C.: All About Maude - A High-Performance Logical Framework: How to Specify, Program, and Verify Systems in Rewriting Logic, Lecture Notes in Computer Science, vol. 4350. Springer, Berlin, Heidelberg (2007). https://doi.org/10.1007/978-3-540-40212-1

7. Durán, F., Eker, S., Escobar, S., Martí-Oliet, N., Meseguer, J., Rubio, R., Talcott, C.: Programming and symbolic computation in Maude. Journal of Logical and Algebraic Methods in Programming **110**, 100497 (January 2020). https://doi.org/10.1016/j.jlamp.2019.100497
8. Eker, S., Martí-Oliet, N., Meseguer, J., Rubio, R., Verdejo, A.: The Maude strategy language. Journal of Logical and Algebraic Methods in Programming **134**, 100887 (August 2023). https://doi.org/10.1016/j.jlamp.2023.100887
9. Escobar, S., López-Rueda, R., Sapiña, J.: Symbolic Analysis by Using Folding Narrowing with Irreducibility and SMT Constraints. In: Proceedings of the 9th ACM SIGPLAN International Workshop on Formal Techniques for Safety-Critical Systems. pp. 14–25. FTSCS 2023, Association for Computing Machinery, New York, NY, USA (Oct 2023). https://doi.org/10.1145/3623503.3623537
10. Gogolla, M., Burgueño, L., Vallecillo, A.: Model finding and model completion with USE **2245**, 194–200 (2018), https://ceur-ws.org/Vol-2245/ocl_paper_9.pdf
11. Jackson, D.: Alloy: a language and tool for exploring software designs. Commun. ACM **62**(9), 66–76 (August 2019). https://doi.org/10.1145/3338843
12. Knuth, D.E.: The Art of Computer Programming. Volume 4a, Part 1: Combinatorial Algorithms. Addison-Wesley, Boston, Mass. ; London (2011)
13. Kuhlmann, M., Hamann, L., Gogolla, M.: Extensive Validation of OCL Models by Integrating SAT Solving into USE. In: Bishop, J., Vallecillo, A. (eds.) Objects, Models, Components, Patterns. pp. 290–306. Springer, Berlin, Heidelberg (2011). https://doi.org/10.1007/978-3-642-21952-8_21
14. López-Rueda, R., Escobar, S., Sapiña, J.: An efficient canonical narrowing implementation with irreducibility and SMT constraints for generic symbolic protocol analysis. Journal of Logical and Algebraic Methods in Programming **135**, 100895 (Oct 2023). https://doi.org/10.1016/j.jlamp.2023.100895
15. Meseguer, J.: Symbolic computation and verification methods in maude. In: Logic-Based Program Synthesis and Transformation - 34th International Symposium, LOPSTR 2025, Rende, Italy, September 9-10, 2025, Proceedings. Lecture Notes in Computer Science, Springer (2025)
16. Meseguer, J., Thati, P.: Symbolic reachability analysis using narrowing and its application to verification of cryptographic protocols. Higher-Order and Symbolic Computation **20**(1), 123–160 (June 2007). https://doi.org/10.1007/s10990-007-9000-6
17. Riesco, A.: Using Narrowing to Test Maude Specifications. In: Durán, F. (ed.) Rewriting Logic and Its Applications. pp. 201–220. Springer, Berlin, Heidelberg (2012). https://doi.org/10.1007/978-3-642-34005-5_11